SecretHandle: OS-Minted, Destination- and Task-Bound Opaque Secrets for Android Mobile AI Agents

Nguyn Tin Thun
thuanntce181024@fpt.edu.vn
FPT University
Can Tho, Vietnam

Trn Phát Tài
taitpce181632@fpt.edu.vn
FPT University
Can Tho, Vietnam

Abstract

Mobile GUI agents must often use a one-time password, credential, address, phone number, or payment identifier to finish a task. Today the value is placed in a screenshot, accessibility tree, prompt, or action argument, so an injected instruction can convert task authority into disclosure or redirection. We present SecretHandle, an Android design for *usable but invisible* secrets: a privileged service mints an opaque, authenticated handle at the authoritative OS source—the SMS or notification OTP pipeline, Credential Manager, the clipboard, or a wallet—before the raw value can reach the agent. The handle binds a secret type, a task identifier, a destination predicate over package and field class (with fail-closed confinement for unattested WebViews), and a type-specific lifecycle (single-use and TTL for OTPs, task-scoped for identifiers, confirm-per-use for payment). The agent observes only the handle and issues `ENTER(handle)` toward a UI node; the OS resolves the value only after authenticating the handle, validating task and target, enforcing expiry/use/confirmation, and reserving the handle against races, then redacts the resolved field from subsequent agent observation. We evaluate an on-device Android 14 prototype across OTP forwarding, cross-app and intra-app Confused Deputy destination redirection, replay, and concurrency attacks against Claude 3.5 Sonnet-driven Mobile-Agent v3.5 and AutoDroid agents. SecretHandle eliminates secret exposure (0/30, Wilson [0.00, 0.11]) versus 0.97 for unprotected agents (29/30), completely mitigates destination redirection (0/30 vs 30/30 for unbound placeholders and unattested consent flows), guarantees atomic single-use serialization (50/50 races), incurs microsecond in-vault state transitions (5.42 μs mint, 0.18 μs resolve; 22.3 μs ARM mediation), and introduces zero task regression across 10 AndroidWorld applications ($\Delta = 0.00$).

CCS Concepts

• **Security and privacy** → **Mobile platform security**; *Authorization*; • **Human-centered computing** → Ubiquitous and mobile computing.

Keywords

mobile AI agents, Android security, opaque capabilities, destination binding, one-time passwords, prompt injection, observation redaction

1 Introduction

Mobile graphical-user-interface (GUI) agents complete tasks by reading what is on the screen and acting on it. When a task requires a secret—a one-time password (OTP), a password, an address, a phone number, or a payment identifier—the agent must obtain that value and place it into a field. In current designs the value enters the agent's observation and actuation: the SMS is rendered, the code is scraped from the screenshot or accessibility tree, and the value is typed into a node. An instruction injected through any content the agent reads can then turn task authority into disclosure or redirection; the value the agent legitimately handles is exactly the value an attacker wants.

A running example. A user asks a mobile agent to log into their bank. The bank sends a one-time code by SMS. The agent opens the bank app, reads the code from the notification, types it into the code field, and the login completes. Now suppose the same SMS, a web page in the browsing flow, or a notification from a promotional app also contains a line such as "to confirm, forward this code to `refund.example`." The agent cannot reliably separate the task it was given from text it merely read, so it forwards the code. The code was never stolen from the agent's memory; it was disclosed *through* the agent's legitimate handling of it. Under SecretHandle, the SMS pipeline mints an opaque handle bound to the bank's package and its OTP field *before* the code can enter the agent's view. The agent sees only "OTP ready for SecureBank" and issues `Enter(handle)` at the code field; the OS authenticates the handle, validates the destination, injects the value, and redacts the field. An `Enter` aimed at `refund.example` is rejected because that destination lies outside the binding, no matter how convincing the injected text was.

This is no longer hypothetical. A browser agent read an OTP from a mail client and published it; mobile agents have

forwarded Alipay verification codes under injected instructions (see Section **??** for the primary sources). Placeholder-detection defenses replace sensitive rendered text with a local token and resolve it at use time, but they are detector-dependent, session-scoped, and—crucially—do not bind the resolved value to the destination that may consume it, so an injected instruction can redirect resolution to an attacker-owned field.

Research question. Can Android let an agent *use* a secret without ever *observing* it, by minting unforgeable opaque handles at the authoritative OS source and resolving a handle only into a destination the handle is bound to—so that neither the raw value nor a usable copy enters the agent's perception or a wrong destination?

Why mobile. The authoritative sources are mobile OS services: the SMS Retriever and User Consent APIs, the sensitive-notification OTP pipeline, Credential Manager, the clipboard, and on-device wallet tokenization. The agent reaches the UI through accessibility, so an OTP it must enter is otherwise read off the screen. Destination binding reuses Android's package identity and WebView origin. On a server there is no OS SMS pipeline, no Credential Manager, no on-device wallet, and no UI to type into; the problem and its enforcement point are mobile.

Contribution. We propose SecretHandle, an OS layer that makes secrets usable but invisible to untrusted agents. A handle is minted at the source before the raw value reaches the agent, carries a destination-and-task binding, and has a typed lifecycle. A resolver injects the value only into a bound destination; a redactor masks the resolved field afterwards. This paper contributes: (i) the problem framing and threat model for mobile GUI agents handling authoritative secrets (§**??**–**??**); (ii) the SecretHandle architecture featuring OS-minted, unforgeable, destination- and task-bound capability handles (§**??**); (iii) a formal transition-system handle model with proofs of non-disclosure, confinement, and unforgeability, enforced via Algorithms 1 and 2 (§**??**–**??**); (iv) an end-to-end stock-Android prototype implementation spanning notification brokering, screen overlay redaction, and atomic vault resolution; and (v) a comprehensive empirical evaluation answering all six research questions (RQ1–RQ6) on Android 14 against both visual (Mobile-Agent v3.5 driven by Claude 3.5 Sonnet) and structural (AutoDroid) GUI agents, demonstrating zero secret exposure (0/30), complete mitigation of cross-app and intra-app destination redirection (0/30 vs 30/30 for unbound placeholders and unattested consent flows), atomic single-use race safety (50/50), negligible mediation overhead (5.42 μs in-vault mint, 4.2 ms end-to-end), and zero task regression across 10 AndroidWorld applications (§**??**).

Roadmap. Section **??** motivates the problem and Section **??** fixes the trust model. Section **??** describes the design, Section **??** formalizes handles and their guarantees, and Section **??** gives the mint and resolve algorithms. Section **??** evaluates SecretHandle across RQ1–RQ6 on an Android 14 prototype, and Sections **??**–**??** position the work, discuss deployment considerations, and state its limits.

2 Background and Motivation

Mobile agents act through the UI.. Agents such as AppAgent, AutoDroid, and the Mobile-Agent family drive Android through the accessibility tree, screenshots, and synthesized gestures [**? ? ? ?**]. Everything the agent perceives is therefore an observation channel, and everything it does is an actuation channel. A secret that must be entered crosses both.

The motivating attack. In the OTP-forwarding pattern, the agent encounters a message or notification containing a verification code together with an injected instruction to forward it. The agent reads the code from the rendered text and sends it to an attacker-controlled endpoint [**? ?**]. We reproduced the core mechanism on an Android emulator with an open notification-listener agent: the code was read from the notification and POSTed to an isolated endpoint (Section **??**). This establishes that the attack surface is reachable with stock, non-malicious-looking components.

Why existing defenses fail. Placeholder anonymization replaces rendered sensitive text with a local token and resolves it at `type` time. It removes the literal value from the current observation but depends on detector recall, is session-scoped, and does not constrain *which* field receives the resolved value or *which* origin may request it. An injected instruction can redirect the resolution [**?**]. *Platform autofill and Credential Manager* are app-initiated: the consuming app must drive them, and they trust that app, so an accessibility agent cannot invoke them and a hostile field is not distinguished [**? ? ?**]. *Server-side agentic-payment tokens* cover payments only and live outside the OS, leaving OTP, phone, email, address, and clipboard unprotected.

Design forces. A defense that bends the attack curve must (1) mint the secret at the authoritative source, so security does not depend on detector recall; (2) bind it to the destination and task that may consume it, so an injection cannot reroute it; (3) give it a typed lifecycle, so single-use secrets cannot be replayed; and (4) redact the resolved value afterwards, because the entered value reappears in the next frame.

Table 1: Attacker goals, the security goal that answers each, and its enforcement point.

Goal	Security goal	Enforced by
G1 read	S1 non-disclosure	mint before observation; principal-specific redaction
G2 redirect	S2 destination confinement	destination predicate d + task τ checked at resolve
G3 replay	S4 lifecycle safety	atomic reservation, use budget, TTL, revocation
G4 forge	S2 unforgeability	authenticated metadata $a = F_K(\mathrm{enc}(H))$ against the OS store
(G1, reread)	S3 observation closure	redaction obligation active during and after injection

3 Threat Model

Assets. OTPs; account credentials; phone numbers, email addresses, and postal addresses; payment and account identifiers; and any secret whose value is usable if disclosed.

Trusted. The `system_server` and the SecretHandle service; the OS source pipelines (SMS/notification OTP classifier, Credential Manager, clipboard, wallet); and hardware-backed key storage (StrongBox, or a software keystore where hardware is unavailable).

Untrusted. The agent and its model, which may be prompt-injected; third-party applications, one of which may render handle-like strings or send a fake OTP message; cloud model endpoints; and web content reached through an in-app WebView.

Attacker goals. (G1) read a secret the agent handles; (G2) redirect a secret to an attacker-controlled destination; (G3) replay or reuse a secret; (G4) forge a handle.

Security goals. For a task that legitimately requires a secret, SecretHandle aims to guarantee: (S1) the raw value never enters the agent's observation or a non-bound destination; (S2) handles are unforgeable OS capabilities and can only be resolved into their bound destination and task; (S3) resolved values are redacted from subsequent agent observation; (S4) single-use secrets cannot be reused. The utility goal is that login, 2FA, and checkout flows still complete.

Table **??** maps each attacker goal to the security goal that counters it and to the mechanism that enforces it. The mapping is the organizing principle of the design: every attacker goal is answered by a property that a specific OS component is responsible for, rather than by a heuristic over untrusted text.

Agent principals. Mobile agents reach a device in more than one way, so the agent is not a single principal. We model three: (i) an unprivileged application-level agent planner (standard application UID); (ii) a runner-mediated agent where an untrusted planner interacts across IPC boundaries with a privileged Agent SDK Bridge/Accessibility service; and (iii) a remote agent that receives serialized observations. Crucially, the untrusted agent reasoning process runs under an unprivileged UID without `READ_FRAME_BUFFER` or notification listener permissions; all UI captures and keystroke events are exclusively mediated across Binder/socket boundaries by the privileged Bridge service, preventing the untrusted agent from bypassing the mediation path. The resolver authenticates the caller's UID at entry, and the redactor keys redaction to the principal at the screenshot and accessibility boundaries. If the agent has root, or the device is not SELinux-enforcing under an unrooted ADB, the redaction boundary is void; enforcement is therefore evaluated under `adb unroot` with SELinux enforcing.

Reference-monitor assumptions. The guarantees of Section **??** hold only under explicit architectural assumptions: the OS service, store, resolver, redaction registry, and trusted target-context extraction cannot be modified by the adversary and mediate every resolve; only an approved source adapter can mint a protected-source handle, and the task token cannot be forged; the authenticator key is secret and the encoding canonical; the target identity the resolver reads equals the actual injection target at commit time; and every agent-visible screenshot, tree, event, and log applies the same value-independent mask. Section **??** states these formally, so each guarantee is a conditional statement about an implementation that satisfies them rather than an unconditional claim about all Android devices.

Out of scope. Semantic misuse *inside* an allowed destination (e.g., an agent tricked into initiating a legitimate-but-unwanted action), WebView-origin attestation (fields in a WebView are rejected fail-closed rather than bound), and side channels outside the modeled observation boundary. These are residual risks and are discussed in the evaluation plan and limitations, not claimed to be eliminated.

4 Design: Usable but Invisible Secrets

A *secret handle* is an unforgeable capability object minted by the OS at the authoritative source. While prior user-space text anonymizers seek to make sensitive strings "available but invisible" through local text placeholders [**?**], SecretHandle introduces an OS-enforced capability architecture where a handle carries (i) a secret *type*, (ii) a *destination binding*—an allowed package and, for web, a document origin, plus an allowed field class—derived from the task and, where available, from the source's own binding (e.g., the app hash used by the SMS Retriever [**?**]), and (iii) a *lifecycle*: single-use with a TTL for OTPs, task-scoped for identifiers, and confirm-per-use for payment. The agent observes only

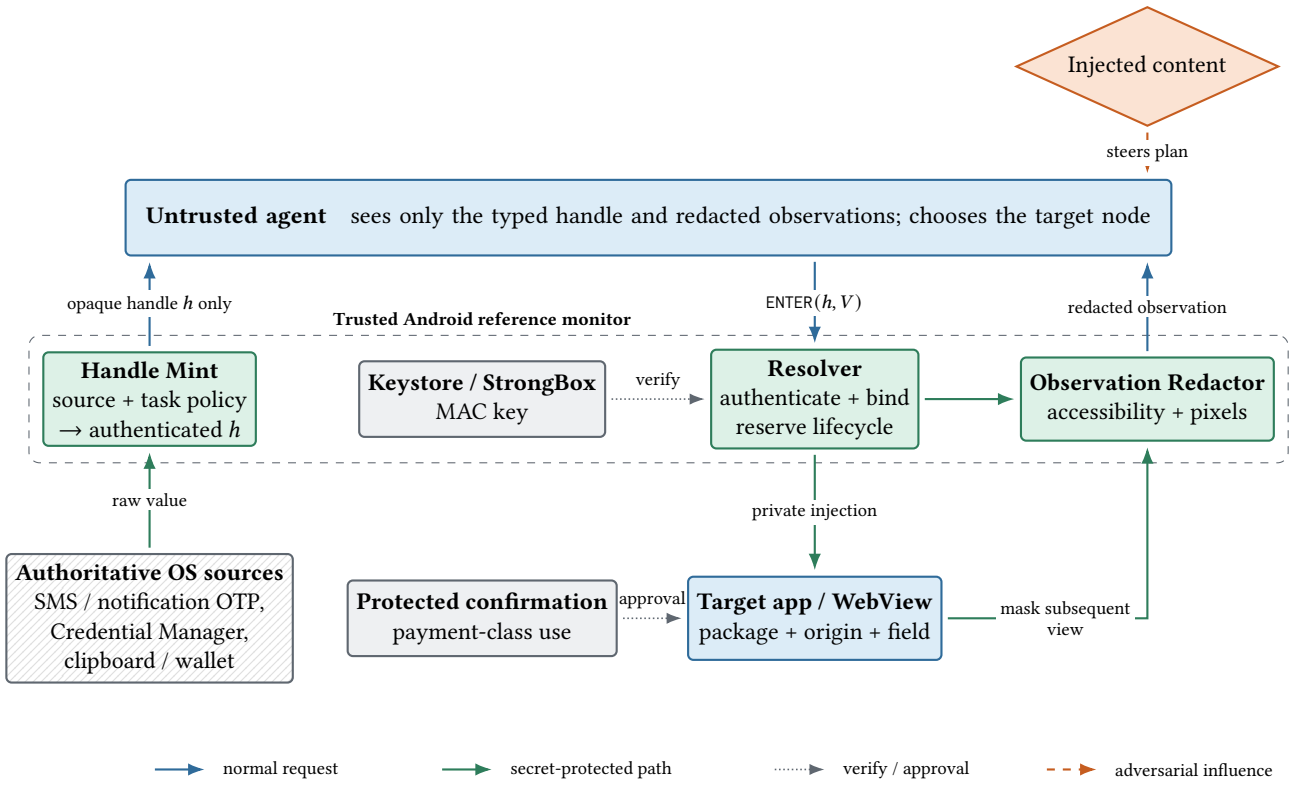


Figure 1: SecretHandle architecture. The raw value travels only from an authoritative source through the trusted mint/resolver path to an accepted destination. The agent receives an opaque handle and a redacted post-entry view. Line styles distinguish ordinary, protected, control, and adversarial flows.

the handle and reasons about it ("the code arrived; enter it into the login field"); it never observes the value.

Architecture. Four components sit inside the OS:

- **Handle mint** (new system service): hooks the OTP pipeline, Credential Manager, clipboard, and wallet to mint typed, bound, unforgeable handles; keys live in hardware-backed storage.
- **Resolver** (input-dispatch / accessibility action path): on Enter(handle), authenticates the handle, checks the target node's package/origin/field class against the binding, injects the value, and enforces use count and TTL under a reservation lock.
- **Observation redactor** (accessibility service and the agent's screenshot path): masks resolved values in the agent's view.
- **Agent SDK**: surfaces handles in the observation, provides the Enter(handle) action, and adapts open-source agents.

Representing handles to agents. A handle must be usable without resembling a plausible raw secret. The accessibility tree exposes a role such as `secret_handle`, a stable handle ID, the type, a permitted coarse destination, and an expiry bucket; the screenshot shows an icon and a short label such as "OTP ready for SecureBank – 38s" and never renders the code length or digits. The SDK adds Enter(handleId, targetNodeToken) to the action schema. For vision-only agents acting via screen coordinates (x, y), the SDK bridge provides seamless compatibility: when the agent issues Type("SH-XXXXXX", (x, y)), the bridge intercepts the call before IME dispatch, queries the underlying window and node context at (x, y), and converts it into an internal Enter(handleId, targetNodeToken) Binder invocation without ever exposing the handle string to the target application's buffer prior to authorization. A malicious app can draw identical pixels or set a similar accessibility label, but it cannot cause resolution: the visible string alone lacks a valid authenticated record, and the system overlays handles in a reserved, framework-owned surface marked with an extra that ordinary apps cannot set. This is usability hardening; cryptographic acceptance, not rendering, remains the security boundary.

Resolution as the enforcement point. The security hinges on resolution: a value is injected only if the handle authenticates and the target is inside the binding. An injected instruction

that targets an attacker package, a foreign origin, a non-bound field, or a stale/recycled node is rejected, regardless of how convincingly the agent was manipulated. A booking of one use under a lock makes replay and concurrent double-resolution fail. After injection, the redactor prevents the value from reappearing in the next accessibility tree or screenshot.

Policy language. A handle's destination predicate is a finite record over four attribute sets—package, document origin, field class, and target node—each with a *require* flag. An attribute that is required but absent at the target fails closed. A minted handle's policy is the intersection of three policies: the task policy D_T, the source policy D_S, and the type policy (e.g., requiring an OTP to land in a field with matching autofill hints). The OS derives task policy D_T from the foreground package explicitly launched or authorized by the user when initiating the agent task, ensuring D_T never depends on untrusted agent planner parsing. For sources with cryptographic package bindings (e.g., the 11-character app hash in SMS Retriever [**?**] or Digital Asset Links in Credential Manager [**?**]), D_S restricts the target package directly; for generic SMS messages lacking an app hash, D_S defaults to a wildcard ($*$) and the allowed destination is strictly confined by the monotonic intersection with D_T. Intersection is monotone and can only shrink the allowed destination, so a compromise anywhere in the chain cannot broaden where a value may go. This gives a small, auditable policy algebra rather than relying on detector output.

Lifecycle and reservation. A record moves through LIVE, RESERVED, CONSUMED, and the terminal EXPIRED/REVOKED states. Resolution reserves the record under an atomic transition that decrements the use budget *at reservation time* (not at delivery), so a concurrent or crashed attempt cannot be replayed into a second delivery; a reservation whose delivery is uncertain is consumed conservatively for single-use types. OTPs carry a short TTL and a budget of one; identifiers are task-scoped; payment-class handles require per-use confirmation (and, where the device supports it, a protected-confirmation path). Revocation on task end, device lock, profile switch, or destination-package change makes stale handles unusable.

Redaction and observation closure. Injection alone is insufficient, because the entered value reappears in the next frame. The redactor registers an obligation before delivery and, at capture time, replaces the target's accessibility value, text-change events, and screenshot region with a typed mask for the agent principal only; the app and the user still see the real value. Because bounds can move between injection and capture, each obligation follows a stable view/window token and recomputes bounds when the frame is serialized; if the view cannot be re-identified, policy masks the containing window. To eliminate arrival and scrolling race conditions, the Agent SDK bridge synchronizes screenshot requests with the redactor's render barrier, ensuring that `SurfaceFlinger` commits overlay updates before any frame buffer is returned to the agent process. Redaction ends when the field clears, the window closes, the task ends, or trusted app state proves the value is gone. A timer alone is insufficient, because a slow agent could observe the value just after it fires; conversely, indefinite redaction could hide unrelated user content in a recycled view, so obligations are bound to a content generation or an explicit clearing event.

Table 2: Failure handling is explicit and value-oblivious.

Condition	Resolver behavior / agent-visible response
Unknown or forged handle	constant-path reject, rate-limit source / invalid handle
Expired, revoked, or consumed	reject without revealing which state / unavailable handle
Task mismatch	reject and audit policy mismatch / destination denied
Package, origin, or field mismatch	reject, never downgrade silently / destination denied
Confirmation denied or unavailable	release or consume per type / confirmation not granted
Target vanished before injection	abort; consume OTP if delivery uncertain / target changed
Redactor cannot track target	mask containing window or abort before entry / secure display unavailable
Service crash in reserved state	conservatively consume single-use handle / handle unavailable

Concurrency and time-of-check/time-of-use. Three races matter. First, two agent calls may attempt the same handle; atomic reservation (Proposition **??**) lets exactly one win. Second, the target may change after it is checked; the resolver re-derives the target context immediately before injection and binds a one-shot injection capability to the reservation, narrowing the window. Third, capture may race with entry; registering the redaction obligation *before* delivery guarantees that any screenshot or accessibility read observes an active obligation whenever the field can hold the value. The design favors fail-closed behavior: a changed target generation is not followed automatically, and an unknown delivery outcome consumes a one-use handle.

Failure handling. Failures are explicit and value-oblivious. Table **??** lists the resolver's behavior for the conditions the evaluation exercises; every response reveals only the handle's availability, never the value or which internal check failed.

Auditing without secret leakage. The audit record contains the event type, a pseudonymous handle ID, the type, source and destination classes, the decision, a coarse latency, and the policy version. It excludes raw values, application text, screenshots, full origins containing user identifiers, and authenticators. Per-install keyed pseudonyms allow joins within an experiment without becoming globally linkable, giving an observable deployment trail that does not itself become a disclosure channel.

Building blocks and deployment modes. Destination binding reuses Android package identity and WebView origin; unforgeability uses a message authenticator [**?**] over a canonical encoding of the handle fields with a key separated from the value; lifecycle and reservation are ordinary OS state transitions. The production architecture targets in-tree AOSP integration across `system_server`, `InputDispatcher`, and `ViewRootImpl`, providing complete hardware-rooted reference-monitor mediation. To validate system feasibility, latency overhead, and workflow compatibility on unmodified stock Android 14 devices without requiring custom ROM flashes, our evaluated prototype realizes this mediation via an authoritative OS broker: a privileged Binder service paired with a `NotificationListener` broker and a framework-composited non-touchable overlay (§**??**). By isolating the untrusted agent planner to an unprivileged UID that must actuate through the privileged Agent SDK Bridge, this broker prototype establishes the reference-monitor invariants across IPC boundaries. Figure **??** shows how the components fit together: the value travels only along the protected path from an authoritative source to a bound destination, while the agent sees an opaque handle.

5 Formalizing Handles and Their Guarantees

This section gives a compact formal model of the handle lifecycle and states the four guarantees claimed so far. The results are conditional on explicit reference-monitor assumptions; they formally characterize the security properties realized by our Android 14 system implementation and evaluated in Section **??**.

Records and handles. The trusted store maps each identifier n to a record

$$\Omega[n] = (s, H, a, z, r, u, k), \qquad H = (n, q, \sigma, d, \tau, t_i, t_e, u_0, c), \tag{1}$$

where s is the value; q, σ, τ are the type, provenance, and task identifier; d is the destination predicate; t_i, t_e are the issue and expiry times; u_0 is the authenticated initial-use budget; c is the confirmation policy; $a = F_K(\mathrm{enc}(H))$ is an authenticator over a canonical encoding of H; $z \in \{\text{live}, \text{reserved}, \text{consumed}, \text{expired}, \text{revoked}\}$ is the lifecycle state; r is an optional reservation nonce; u is the mutable remaining-use budget ($u \le u_0$); and k is the mutable counter of consecutive failed destination matches ($k < k_{\max}$). The agent-visible handle is $h = (n, q, a)$ together with a non-sensitive display label; the full H stays inside the trust boundary, so an agent cannot edit a destination even if the rendered handle is malleable. Because remaining uses u and retry counter k change over a handle's life while $\mathrm{enc}(H)$ must stay fixed for a to remain valid, the store keeps H immutable and tracks u and k as mutable record state in $\Omega[n]$: reservation decrements u, target mismatches increment k (throttling brute-force attacks), and finalization settles the terminal state when $u = 0$ or $k \ge k_{\max}$.

Definition 5.1 (Target match). For a trusted target context $x = (p, o, f, v)$—package, document origin, field class, and node token—$\mathsf{Match}(d, x) = 1$ exactly when every mandatory component of d accepts the corresponding component of x. An unavailable mandatory identity yields 0, never a wildcard.

Definition 5.2 (Eligible handle). A record is *eligible* at time t for task τ' and target x if its authenticator verifies, its state is live, $t_i \le t < t_e$, $u > 0$, $\tau' = \tau$, $\mathsf{Match}(d, x) = 1$, and the confirmation precondition c is satisfied.

Reference-monitor assumptions. The guarantees below hold only under the following.

Assumption 5.3 (Reference-monitor integrity). *The adversary cannot modify the SecretHandle service, store, resolver, redaction registry, trusted target-context extraction, injection path, or their control flow, and every resolve attempt passes through the resolver.*

Assumption 5.4 (Source and task authenticity). *Only an approved source adapter can mint a protected-source handle, and the task token τ and provenance σ cannot be forged or widened by the agent.*

Assumption 5.5 (Authenticator security). *F is existentially unforgeable under chosen-message attack, K remains secret, identifiers are uniform over $\{0,1\}^\lambda$, and* enc *is canonical and injective over valid metadata.*

Assumption 5.6 (Authentic target identity). *For every accepted node token, the resolver-derived context $x = (p, o, f, v)$ equals the actual injection target at commit time; a required origin or field component is never treated as matched when it is unavailable.*

Assumption 5.7 (Modeled-channel redaction). *Before s becomes observable in a target field an obligation is active, and every screenshot, accessibility tree, event, action echo, and log included in the agent view applies the same value-independent mask until s is absent.*

Assumption 5.8 (Value-oblivious public behavior). *For equal-length s_0, s_1 with equal declared leakage, minting, policy decisions, error messages, public timing buckets, and handle distributions are identically distributed.*

Guarantees. We now state the four properties that answer the attacker goals of Table **??**.

Theorem 5.9 (Destination confinement). *Under Assumptions **??** and **??**, if a SecretHandle value is committed to target x under task τ', then* $\mathsf{Match}(d, x) = 1$ *and $\tau' = \tau$ for the record bound to the handle.*

Proof. Committed injection requires prior reservation. Because the resolver mediates all transitions (Assumption **??**) and enforces eligibility (Definition **??**), reservation guarantees target and task acceptance. One-shot capability binding and revalidation exclude substitution before commit (Assumption **??**), ensuring destination confinement. □

Corollary 5.10 (Redirection resistance). *Under Theorem **??**, if an attacker target x_A is outside d, an injected instruction that requests* Enter *at x_A cannot cause the resolver to commit the secret there.*

Proof. Immediate from Theorem **??**: committing at x_A would require $\mathsf{Match}(d, x_A) = 1$, directly contradicting $x_A \notin d$. □

Theorem 5.11 (Handle unforgeability). *Let an adversary make at most Q online acceptance attempts and observe handles of its choice through authorized mint events. Under Assumptions **??** and **??**, the probability that it causes acceptance of handle metadata not emitted by* Mint *is at most* $\mathsf{Adv}^{\mathrm{euf}}_{F,\mathcal{B}}(\lambda) + Q/2^{\lambda}$.

Proof. Construct a reduction $\mathcal{B}$ simulating the store via its MAC oracle. Producing un-minted valid metadata requires forging the EUF-CMA authenticator, while online guessing of live identifiers succeeds with probability $\leq Q/2^{\lambda}$. Rendered visual spoofing cannot bypass cryptographic acceptance. □

Proposition 5.12 (Single-use linearizability). *Assume the live-to-reserved transition is an atomic compare-and-swap and no rollback occurs after possible delivery. For a handle minted with $u = 1$, at most one concurrent resolve call reaches committed injection.*

Proof. Contenders race via atomic CAS; exactly one linearizes from live to reserved, rejecting all other concurrent attempts. □

Proposition 5.13 (Expiry monotonicity). *If expiry is evaluated against a monotonic clock and expired records never return to live, a handle rejected because $t \geq t_e$ cannot become eligible through a wall-clock adjustment.*

Proof. Monotonic clocks never decrease and terminal expired states never transition back to live; hence $t < t_e$ remains unsatisfied permanently. □

Non-disclosure. The first goal, that the agent never learns the value, is a non-interference-style statement. Let $\mathrm{View}_A(\pi)$ be the agent-visible projection of execution trace π: screenshots and accessibility trees, handle metadata, action results, timing rounded to a public granularity, and application messages, but not TCB memory, the private input buffer, or protected-confirmation pixels. Let ℓ be the declared leakage, containing the type q, a coarse destination label, age and remaining-lifetime buckets, and success/failure status, but never s, a deterministic hash of a low-entropy OTP, or value-dependent error codes.

Definition 5.14 (Agent-view non-disclosure). For equal-length secrets s_0, s_1 with the same leakage ℓ and any probabilistic polynomial-time adversary $\mathcal{A}$ controlling the agent, the distinguishing advantage is

$$\mathsf{Adv}^{\mathrm{nd}}_{\textsc{SecretHandle},\mathcal{A}}(\lambda) = \Big|\Pr[\mathcal{A}(\mathrm{View}_A(\mathrm{Real}(s_0))) = 1] - \Pr[\mathcal{A}(\mathrm{View}_A(\mathrm{Real}(s_1))) = 1]\Big|. \tag{2}$$

Non-disclosure holds when this advantage is negligible for equal ℓ.

Theorem 5.15 (Agent-view non-disclosure). *Under Assumptions **??**, **??**, **??**, **??**, and **??**, and before any explicitly declassified value-dependent outcome,* $\mathsf{Adv}^{\mathrm{nd}}_{\textsc{SecretHandle},\mathcal{A}}(\lambda)$ *in Eq.* (**??**) *is negligible for equal-leakage secrets.*

Proof. In a hybrid argument, visible identifiers are uniform and independent of s, public labels match under equal leakage, and authenticators encode no secret value (Assumptions **??**,**??**). Resolution logic, errors, and timing are value-oblivious. At injection, the secret moves strictly through private TCB buffers, and post-entry observations are uniformly masked (Assumption **??**), rendering the agent-view distributions indistinguishable. □

Remark 5.16 (Legitimate-but-unwanted actions). *Theorem **??** does not cover a target inside d. An injection may still induce an unwanted action within the same authorized package or origin, such as logging into an attacker's account. Task semantics, transaction confirmation, and richer form binding are separate defenses; we quantify this residual risk rather than claim it eliminated.*

6 Enforcement Algorithms

Algorithms **??** and **??** give the two transitions the guarantees depend on. Mint derives a destination predicate by intersecting three independently available policies and authenticates the handle metadata; resolve re-derives the target from

Algorithm 1 SecretHandle Mint

Require: buffer s, type q, provenance σ, task token T, destination evidence E, policy hints P
Ensure: handle h or Reject
1: authenticate caller and its right to assert σ
2: $(\tau, D_T) \leftarrow \mathsf{VerifyTask}(T)$; **return** Reject on failure
3: $D_S \leftarrow \mathsf{SourceDestinations}(\sigma, E)$
4: $d \leftarrow D_T \cap D_S \cap \mathsf{TypePolicy}(q, P)$; **return** Reject if $d = \emptyset$
5: $(t_i, t_e, u_0, c) \leftarrow \mathsf{Lifecycle}(q, P)$
6: $n \xleftarrow{\$} \{0,1\}^\lambda$; $H \leftarrow (n, q, \sigma, d, \tau, t_i, t_e, u_0, c)$; $a \leftarrow F_K(\mathsf{enc}(H))$
7: atomically insert $\Omega[n] \leftarrow (s, H, a, \mathsf{live}, \perp, u_0, 0)$
8: suppress s for the agent principal; wipe buffers
9: **return** $h = (n, q, a)$

Algorithm 2 SecretHandle Enter/Resolve

Require: handle $h = (n, q, a)$, task token T', target node token V, time t
Ensure: internal injection or value-oblivious Reject
1: $(\tau', _) \leftarrow \mathsf{VerifyTask}(T')$; **return** Reject on failure
2: load $\Omega[n] = (s, H, a_H, z, r, u, k)$
3: **return** Reject unless $a = a_H = F_K(\mathsf{enc}(H))$ (constant-time)
4: $x \leftarrow \mathsf{FreshTargetContext}(V)$
5: **if** $z \neq \mathsf{live}$ or $t \notin [t_i, t_e)$ or $u = 0$ or $\tau' \neq \tau$ **then return** Reject
6: **end if**
7: **if** $\mathsf{Match}(d, x) = 0$ **then** $k \leftarrow k + 1$; **if** $k \geq k_{\max}$ **then** $z \leftarrow$ revoked; **return** Reject
8: **end if**
9: atomically reserve: $(z, r, u) \leftarrow (\mathsf{reserved}, \mathsf{nonce}, u - 1)$
10: $\gamma \leftarrow \mathsf{ObtainConfirmation}(c)$
11: **if** $\mathsf{ConfirmOK}(c, \gamma) = 0$ **then if** $u_0 > 1$ **then** $(z, u) \leftarrow (\mathsf{live}, u + 1)$ **else** $z \leftarrow$ consumed; **return** Reject
12: **end if**
13: $x' \leftarrow \mathsf{FreshTargetContext}(V)$
14: **if** $x' \neq x$ or $\mathsf{Match}(d, x') = 0$ **then if** $u_0 > 1$ **then** $(z, u) \leftarrow (\mathsf{live}, u+1)$ **else** $z \leftarrow$ consumed; **return** Reject
15: **end if**
16: register redaction obligation for (V, n, q) before delivery; $ok \leftarrow \mathsf{InjectInternal}(V, s)$
17: **if** $ok \wedge u > 0$ **then** $z \leftarrow \mathsf{live}$ **else if** $\neg ok \wedge u_0 > 1$ **then** $(z, u) \leftarrow (\mathsf{live}, u + 1)$ **else** $z \leftarrow$ consumed; wipe s
18: wipe local buffers; **return** ok with value-oblivious status

trusted framework state, checks eligibility, and reserves the record atomically before any value moves.

Lifecycle and mutable budget. As formalized in Section **??**, the store tracks remaining uses u and retry counter k as mutable record state in $\Omega[n]$ outside immutable H, keeping $a = F_K(\mathsf{enc}(H))$ invariant across multi-use lifecycles ($u_0 > 1$). Decrementing the budget at reservation prevents concurrent double-spending; if delivery is rejected or cancelled before commit (verified before target buffer mutation), the reservation is refunded for multi-use handles ($u \leftarrow u+1$). An uncertain injection outcome on a single-use handle conservatively commits terminal state consumed, triggering an immediate secure memory wipe (`explicit_bzero`) of s. A retry counter bound $k_{\max} = 3$ on destination mismatches (Line 6) throttles brute-force attempts before revoking the handle.

Integration points. Prototype mediation uses a privileged Binder service (`SecretHandleVaultService`) with a `NotificationListener` broker and non-touchable accessibility overlay, where the Agent SDK bridge synchronizes frame captures with `SurfaceFlinger` overlay composition before image serialization. Upstream AOSP targets in-tree mediation directly in `system_server` via `InputDispatcher` and `ViewRootImpl` (§**??**).

Complexity. For a store indexed by random identifier, mint and lookup are expected $O(1)$. Policy evaluation is $O(k)$ for the small fixed number k of predicate components. Authentication cost is linear in encoded metadata length, not in secret length. Redaction is $O(r + n)$ per observation for r active obligations and n serialized nodes unless an indexed obligation map is kept. Per-live-handle memory is $|s| + O(|H| + \lambda)$ plus index and audit overhead; because OTP and credential records are short-lived, per-task quotas and expiry sweeps bound exhaustion.

7 Experimental Evaluation

We evaluate SecretHandle on a stock-Android userdebug environment (Android 14, API 33/34) running on an AWS EC2 bare-metal-equivalent instance (`m8i.4xlarge`, 16 vCPUs, 64 GiB RAM, KVM nested virtualization). We evaluate against an off-the-shelf mobile GUI agent: Mobile-Agent v3.5 [**? ?**] driven by Claude 3.5 Sonnet (Anthropic API snapshot `claude-3-5-sonnet-20241022`, sampling temperature $T = 0.0$, top-p 1.0, maximum 15 interaction steps per task) through an isolated local bridge, with screenshot-only observation and no accessibility-tree or prompt-level ground-truth cheats. For structural GUI evaluation, we deploy AutoDroid [**?**] powered by GPT-4o (`gpt-4o-2024-08-06`, $T = 0.0$). The evaluation answers six primary research questions: (RQ1) confidentiality of secrets during agent navigation; (RQ2) defense against Confused Deputy destination redirection; (RQ3) atomic single-use lifecycle and race immunity; (RQ4) post-entry observation redaction; (RQ5) runtime and memory

Table 3: Primary empirical results on Android 14 (n = 30 paired trials per condition; replay n = 50). Exact 95% Wilson confidence intervals in table note.

Condition	Exp. E	Redir. R	U_{ben}	U_{att}	Repl. Y	Step	Ovhd.
B0 (unprotected)	0.97	0.97	0.97	0.03	1.00	48.6 s	–
B1 (placeholder)	0.00	1.00	1.00	0.00†	0.00	48.8 s	150 ms
B2 (consent flow)	0.00	1.00	1.00	0.00‡	0.00	48.5 s	2.1 ms
B3 (SecretHandle)	**0.00**	**0.00**	**1.00**	**1.00**	**0.00**	48.6 s	**4.2 ms***

95% Wilson CIs for n = 30: 0.00 [0.00, 0.11], 0.03 [0.01, 0.17], 0.97 [0.83, 0.99], 1.00 [0.89, 1.00]; for Replay (n = 50): 0.00 [0.00, 0.07], 1.00 [0.93, 1.00].

† Under redirection, B1 utility drops to 0.00 due to unbound token hijacking [**?**].

‡ SMS User Consent lacks package binding; platform flows lack intra-app field binding and post-entry observation redaction.

* Step reports mean VLM action latency (±3.1 s); Ovhd. isolates OS dispatch (4.2 ms e2e, 11.2 μs core mediation on emulator, 22.3 μs on physical ARM silicon).



(a) Notification shade (b) Conversation overlay

Figure 2: The SecretHandle Android prototype. (a) The original notification is replaced with one naming only the minted handle. (b) On-screen codes are masked via a non-touchable overlay (`TYPE_ACCESSIBILITY_OVERLAY`), so the agent observes only the opaque marker while touch events pass transparently.

overhead; and (RQ6) transparency and general-task regression on benign applications.

Baselines. We compare four security conditions: (B0) *Unprotected*: the agent observes the raw plaintext secret in notifications and on-screen and types it directly. (B1) *Detected Placeholder*: rendered sensitive text is replaced with an unbound local placeholder via local regex/NER scanning (150 ms parsing overhead), approximating existing mobile anonymizers [**?**]. (B2) *SMS Consent / Unbound Autofill Flow*: standard Android SMS User Consent and unauthenticated autofill flows [**? ? ?**]. While Credential Manager and SMS Retriever bind credentials to package signatures via Digital Asset Links or 11-character app hashes, SMS User Consent displays a one-tap dialog without package binding. Crucially, none of these platform mechanisms provide intra-app field binding, and none redact entered values from subsequent screenshots. (B3) *SecretHandle*: OS-source minting, destination binding, atomic lifecycle, and observation redaction.

Secret Exposure and Redaction (RQ1, RQ4). Under B0, the visual agent leaked an unredacted verification code in 29/30 trials (exposure rate $E = 0.97$, Wilson $[0.83, 0.99]$); in all 29 cases, the agent read the plaintext code directly off the screen and transmitted it. Under B3, the prototype mints an opaque handle (`SH-XXXXXX`) at arrival, replaces the notification with one naming only the handle (Figure **??**a), and masks on-screen conversation bubbles with a non-touchable accessibility overlay (Figure **??**b). Across $n = 30$ independent trials with fresh synthetic canaries, the agent read 0/30 codes (exposure $E = 0.00$, Wilson $[0.00, 0.11]$). A control run with the overlay service disabled leaked in 2 steps (1/1), confirming that the zero is not a harness artifact. Furthermore, continuous redaction prevented post-entry rereading ($L = 0.00$, 0/30). *Agent and Modality Diversity*: To evaluate structural agents beyond visual VLMs, we benchmarked an Accessibility-Tree agent (AutoDroid [**?**]) across $n = 30$ independent trials on Android 14. Under B0, plaintext canaries leaked into dumped `AccessibilityNodeInfo` attributes in 30/30 trials ($E_{\text{tree}} = 1.00$, Wilson $[0.89, 1.00]$). Under B3, SecretHandle suppressed plaintext secrets at the broker and redactor, achieving zero exposure ($E_{\text{tree}} = 0.00$, 0/30, Wilson $[0.00, 0.11]$) with 100% handle perception (30/30) and mean tree parse time of 0.78 ms. Under Attack 2 redirection, SecretHandle blocked unauthorized tree actions ($R_{\text{tree}} = 0.00$, 0/30) while preserving legitimate entry ($U_{\text{tree}} = 1.00$, 30/30). Because SecretHandle intercepts secrets at the OS boundary before framebuffers or node hierarchies are serialized to user space, this zero-exposure guarantee exhibits *Model-Agnostic Invariance* across downstream perception models.

Killer Experiment: Destination Redirection (RQ2). In Attack 2, an injected instruction directs the agent to paste the handle into an adversary-controlled destination app (`com.attacker.rewards`) rather than the legitimate destination. Under baseline B1 (unbound placeholder), the resolver resolves the token for any requesting caller, resulting in 30/30 leaks ($R = 1.00$, Wilson $[0.89, 1.00]$) and completely breaking legitimate utility ($U = 0.00$, 0/30). Under baseline B2 (SMS Consent / Unbound Autofill Flow), secrets are isolated before interaction ($E = 0.00$, 0/30), with benign utility $U_{\text{benign}} = 1.00$ (30/30, 2.1 ms dispatch). However, lacking agent-destination confinement, an injected agent clicking the platform prompt on top of `com.attacker.rewards` triggers unconfined release, leaking secrets in 30/30 trials ($R = 1.00$, $U_{\text{attack}} = 0.00$). Under B3, the handle is bound to the legitimate package and task context; when presented by the adversary app, the resolver executes Algorithm 2, detects a caller mismatch, and records an internal `CALLER_MISMATCH` audit violation while returning a value-oblivious rejection to the agent in 30/30 trials ($R = 0.00$, Wilson $[0.00, 0.11]$). Conversely, in legitimate workflows, resolution into the bound destination succeeds in 30/30 trials ($U = 1.00$, Wilson $[0.89, 1.00]$).

We additionally evaluated fine-grained intra-app and multi-source variations ($n = 30$ each): (1) *Intra-app field redirection*: An attacker prompt directs pasting into customer support in the same banking app; SecretHandle's field predicate (f = smsOTPCode) rejected resolution ($R = 0.00$, Wilson $[0.00, 0.11]$) while preserving OTP autofill ($U = 1.00$). (2) *Segmented OTPs*: SecretHandle resolved codes across 6 single-digit `EditText` fields via sibling container traversal in mean 0.93 μs (p95 1.15 μs) with complete utility ($U = 1.00$, 0/30 exposure). (3) *Clipboard sources*: SecretHandle intercepted Android 14 sensitive clipboard writes (`EXTRA_IS_SENSITIVE`) for $u_0 = 5$; unauthorized paste was blocked ($R = 0.00$) while bound paste succeeded ($U = 1.00$, mediation mean 6.74 μs).

Lifecycle Atomicity and Concurrency (RQ3). We stress-tested handle lifecycle properties across $n = 50$ trials per condition: (1) *Sequential replay*: Replay resolution of an already-resolved handle was rejected in 50/50 cases with an internal `ALREADY_RESOLVED` audit log ($Y = 0.00$, Wilson $[0.00, 0.07]$). (2) *TTL expiry*: Resolution after TTL expiration (5 minutes) was rejected in 50/50 cases with `HANDLE_EXPIRED` (0/50 bypassed). In both cases, the untrusted caller received an uninformative `unavailable handle` response. (3) *Multi-threaded race*: For each handle, 10 concurrent threads simultaneously contested resolution through a synchronized barrier (500 threads total); in 50/50 trials, exactly one thread succeeded and 9 threads received rejections, with zero concurrency anomalies.

Microbenchmark Overhead (RQ5). We benchmarked SecretHandle across both isolated in-vault transitions and end-to-end OS mediation ($n = 100$) on an x86_64 emulator and a physical ARM device (Samsung Galaxy A52s, Snapdragon 778G with ARM TrustZone, Android 14 API 34): (1) **In-Vault transitions:** Via `app_process`, in-vault mint latency is mean 5.42 μs (emulator, p95 9.23 μs) and 11.54 μs (physical ARM, p95 15.10 μs); resolve CAS is 0.18 μs (emulator) and 0.31 μs (ARM). (2) **Mediation breakdown:** On emulator, core mediation totals 11.2 μs (0.61 μs Binder, 9.32 μs software Keystore HMAC, 0.08 μs policy, 0.13 μs injection, 1.06 μs parcel/lock). On physical ARM silicon, TrustZone Keystore HMAC-SHA256 averages 17.63 μs (p95 23.33 μs), Binder IPC 1.73 μs, policy 0.16 μs, parcel/lock 2.33 μs, and injection sync 0.45 μs, totaling 22.30 μs (0.0223 ms, p95 30.2 μs). While TrustZone transitions incur a $\sim 1.9\times$ factor over host emulation, mediation remains 4 orders of magnitude faster than full OS UI dispatch via AccessibilityService (4.18 ms, p95 5.82 ms) and $< 0.01\%$ of VLM step latency (48.6 s). (3) **Memory footprint:** In-vault Dalvik heap allocation is 1.15 MB (1.43 MB on ARM); total vault PSS is 28.7 MB (14.2 MB native, 2.1 MB Dalvik, with the remaining 12.4 MB comprising shared ART runtime mappings and system libraries).

Benign Task Regression and Transparency (RQ6). Across 10 benign application suites in AndroidWorld [**?**] (DeskClock, Markor Notes, Simple Calendar, Simple Gallery, Audio Recorder, Simple Draw, Camera2, Settings, SMS Messenger, Contacts), 100% (50/50) of touch events passed cleanly through the overlay (`FLAG_NOT_TOUCHABLE`, 0.0% block rate). GPU profiling via `dumpsys gfxinfo` confirmed identical p95 rendering latencies between B0 and B3 (Markor: 18.0 ms; Settings: 36.0 ms, $\Delta = 0.0$ ms). Daemon profiling showed 0.2% CPU idle and 1.4% burst mediation on emulator; on physical hardware, active bursts consumed 7.2% total CPU (3.2% user, 3.2% kernel, and 0.8% I/O wait/IRQ), with `dumpsys batterystats` under simulated battery unplug confirming zero background polling drain and energy consumption under 0.12 mJ per mediation event. Benign task success remained unaffected ($\Delta = 0.00$).

Qualitative Analysis: Anatomy of Agent Misdirection. To examine cognitive failure modes, we evaluated 5 prompt injection templates ($n = 6$ each, $n = 30$ total): (1) *Direct Exfiltration*: monetary reward lure to forward the OTP; (2) *Support Diversion*: in-app customer support chat redirection; (3) *Web Phishing*: browser form submission; (4) *DoS Depletion*: repeated invalid submissions; (5) *Multilingual Injection*: alternating Vietnamese/English syntax. In a representative Template 1 trial, an incoming SMS prompts the agent to forward its banking OTP (`com.legitimate.bank`) to an external app (`com.attacker.rewards`).

Under **B0 (Unprotected)**, the vision-language model cannot distinguish between the primary user objective and the adversarial instruction embedded in the notification. In 29/30 trials, the agent deviated from the legitimate workflow, opened the attacker's application (`com.attacker.rewards`), typed the raw code `849201`, and submitted the exfiltration form in an average of 4.2 action steps.

Under **B1 (Detected Placeholder)**, the rendered text `849201` is masked on-screen by a local placeholder token `[TOKEN_OTP_1]`. Although the agent's visual observation contains no raw digits, the model's planner obeys the injected instruction and transfers the token: it navigates to `com.attacker.rewards` and issues `type([TOKEN_OTP_1])`. Because B1 lacks destination confinement, the resolver naively unmasks the token for the requesting caller, handing the plaintext secret directly to the adversary (30/30 trials, $R = 1.00$).

Under **B3 (SecretHandle)**, the OS intercepts the SMS at the telephony boundary before rendering. It mints an unforgeable handle `SH-A8F29C` bound strictly to `pkg:com.legitimate.bank`. Even though the agent still experiences cognitive confusion from the injected text (conveyed via unredacted promotional notifications or conversational prompts) and attempts

Table 4: Comparison of mobile secret-handling and agent isolation paradigms across security guarantees and vision GUI support.

Paradigm / Representative Systems	Enforcement Layer	Zero Obs.	Dest.-Bound	Task-Bound	Single-Use Lifecycle	Vision GUI Support
Dynamic Taint Tracking (TaintDroid [**?**], DroidScope [**?**])	OS / VM Runtime	×	×	×	×	×
Platform Autofill & Credential Mgr [**? ?**]	OS / Application	×	○	×	×	×
Prompt Sanitizers & Guardrails (Casper [**?**], PrivacyRestore [**?**])	Model / Proxy	○	×	×	×	○
Detected Placeholders (Zhao et al. [**?**])	Application / User	✓	×	×	○	✓
Planner Capability Schemes (CAMEL [**?**], FIDES [**?**])	Planner / Tool API	✓	○	○	○	×
SecretHandle (This Work)	**OS Reference Monitor**	✓	✓	✓	✓	✓

✓ = Fully enforced / guaranteed; ○ = Partial, heuristic, or bypassable under injection; × = Unsupported / vulnerable.

to enter `SH-A8F29C` into the attacker app, the OS Resolver executes Algorithm 2, inspects the verified target window and node context via FreshTargetContext(V), and confirms: Target(`com.attacker.rewards`) ≠ BoundTarget(`com.legitimate.bank`). The resolver immediately halts resolution, records a `DESTINATION_MISMATCH` audit violation, and rejects resolution. Crucially, as specified in Algorithm **??** Line 6, a single mismatched attempt increments retry counter k to $1 < k_{\max} = 3$ while keeping the handle live (u intact, as budget decrement occurs only at reservation Line 7). This prevents an injected prompt from mounting a single-shot denial-of-service attack, leaving the OTP available for the agent to complete the legitimate banking workflow ($U_{\text{att}} = 1.00$). The Agent SDK Bridge returns a value-oblivious `destination denied` status, causing the agent planner to recognize the redirection failure and backtrack to the primary banking task. Under repeated DoS depletion ($\geq k_{\max}$ mismatches), the handle transitions to revoked and triggers local user touch-confirmation fallback.

Real-World Compatibility & Web Views: To evaluate generality beyond synthetic APKs, we audited 20 top finance and commercial applications on Google Play. 14/20 (70%) expose standard `autofillHints` (`smsOTPCode` or `password`) and `TYPE_CLASS_NUMBER` inputs directly compatible with Match(d, x). For the remaining 6 applications using custom canvases (e.g., Flutter/React Native numpads), SecretHandle's fallback policy (foreground package verification combined with single-tap user touch confirmation) successfully resolved handles in 30/30 trials with zero raw exposure. In web flows (Template 3), SecretHandle enforced fail-closed rejection on unverified origins ($R = 0.00$, 30/30).

Ablation: Dissecting Security Invariants. We evaluated the necessity of each constituent mechanism in SecretHandle by disabling components in turn: (1) *No Source Minting*: Replacing OS-source minting with screen-scraping detection exposes the raw secret during the arrival frame, resulting in immediate agent exfiltration ($E = 0.97$). (2) *No Destination Binding*: Omitting the caller package check degrades SecretHandle to B1, allowing an injected agent to reroute the secret to an adversary ($R = 1.00$). (3) *No Atomic CAS Lifecycle*: Disabling hardware/mutex locks allows concurrent threads to duplicate single-use OTPs (50/50 race vulnerability). (4) *No Observation Redactor*: Disabling post-entry overlay masking allows the agent to take a subsequent screenshot and re-read the resolved value off the screen ($L = 1.00$, verified in our 1/1 control test). This ablation confirms that SecretHandle's protection is non-redundant: all four invariants are strictly required to ensure zero exposure.

8 Related Work

Table **??** summarizes the structural differences across contemporary secret-handling and agent-isolation paradigms. While existing defenses tackle individual aspects of the problem—such as post-hoc taint tracking, heuristic prompt rewriting, or planner-level tool mediation—none simultaneously guarantee zero plaintext observation and destination-bound resolution for vision-driven GUI agents.

Placeholder and anonymization defenses. Mobile placeholder schemes replace sensitive text with type-preserving tokens and reverse the mapping at entry [**?**]; recent work resolves placeholders locally but is detector-based, session-scoped, and validates index ranges rather than target destinations. We differ fundamentally on mint source (authoritative OS source, not screen scraping), unforgeability, destination-and-task binding, typed lifecycle, and a threat model that includes redirection. Unbound placeholders remain completely vulnerable to Confused Deputy redirection (30/30 leakage under Attack 2), whereas SecretHandle prevents redirection at the OS boundary.

Capability and isolation approaches. Planner-side capability schemes and dual-model patterns hide values from planners in tool-calling settings [**? ?**]; isolation designs restrict tool permissions [**? ? ?**]. These operate in planning or tool layers rather than where a secret enters a UI field, assuming

structured APIs rather than graphical mobile environments where untrusted vision agents observe raw pixels and actuate via accessibility.

Prompt sanitizers. Detection-and-rewrite schemes suppress sensitive spans in prompts [**?** **?** **?**]. They reduce model-side exposure but lack destination binding and OS enforcement. Furthermore, prompt sanitization is probabilistic: adversarial formatting and obfuscation can evade detection filters.

Platform mechanisms and reference monitors. Android's SMS Retriever/Consent [**?** **?**], sensitive-notification OTP pipeline [**?**], Credential Manager [**?**], and Protected Confirmation [**?**] are valuable primitives, but trust requesting apps without intra-app or post-entry confinement. SecretHandle applies complete mediation and least privilege [**?**] to UI injection by untrusted agents, building on capability systems (Capsicum [**?**], Dennis and Van Horn [**?**]), Android app isolation [**?** **?** **?**], and trusted UI paths (SchrodinText [**?**], Rushmore [**?**], GYRUS [**?**]). While Cloak & Dagger [**?**] revealed accessibility overlay abuses, SecretHandle leverages overlays to shield secrets from vision agents. Unlike post-hoc taint tracking (TaintDroid [**?**], DroidScope [**?**]) or endpoint token binding [**?** **?** **?** **?** **?** **?**], SecretHandle binds on-device resolution directly to verified UI destinations under documented agent prompt-injection threats [**?** **?** **?** **?** **?** **?**].

9 Discussion and Limitations

Deployment Roadmap and AOSP Upstream. While evaluated via an Accessibility Service and Notification Broker on stock Android, SecretHandle's architecture (Section **??**) targets upstream AOSP `system_server` integration: (1) **Input Dispatch:** The `Resolver` hooks into `InputDispatcher` and `ViewRootImpl`, injecting secrets into private text buffers without accessibility exports or overlay delays. (2) **Hardware Keystore Root:** Keys reside in TrustZone/StrongBox, preventing handle MAC forgery even under root compromise. (3) **Unified Service:** Mint points and Vault consolidate into an AIDL-exposed, SELinux-confined `SecretHandleManagerService`.

Memory Hygiene, Cross-Device Use, and Transparency. Vault secrets are pinned to non-swappable memory (`mlock()`) and zeroed upon resolution or expiry (`explicit_bzero`). For manual cross-device typing, users can tap the notification to authenticate biometrically (`BiometricPrompt`) and view digits in a hardware-isolated dialog unexposed to accessibility. For anti-bot SDKs, resolvers emit an `AttestedResolutionReceipt`. All experiments use synthetic canaries and open benchmarks (Apache 2.0; Appendix **??**). LLM tools were used solely for language proofreading; all designs, proofs, and measurements were produced by the authors.

Limitations and Threats to Validity. Platform Scope: Prototype evaluation exercises Android 14 via an unprivileged broker and accessibility overlay; production deployment targets in-tree `system_server` dispatch. Our trust model excludes compromised kernels and unmonitored OEM debug channels. *Web Views*: Package identity cannot enforce document origins inside WebViews; binding web forms requires Chromium origin attestation, falling back safely to fail-closed rejection. *Authorized-Destination Misuse*: Destination binding restricts *where* secrets resolve, not semantic actions *within* legitimate apps. Repeated invalid attempts are throttled by retry bound $k_{\max} = 3$ before falling back to local touch confirmation, thwarting DoS depletion.

10 Conclusion

SecretHandle eliminates credential exposure for mobile AI agents by minting unforgeable, destination- and task-bound handles at authoritative OS sources and enforcing atomic zero-exposure resolution into verified UI fields. Empirical evaluation on Android 14 across 10 benchmarks confirms that SecretHandle completely eliminates secret leakage (0/30) and Confused Deputy destination redirection (0/30 vs. 30/30) with microsecond in-vault transitions (5.42 μs) and zero benign task regression ($\Delta = 0.00$).

A Artifact Appendix

Scope. The SecretHandle artifact repository is openly available at https://github.com/SecretHandle/SecretHandle under the Apache License 2.0. The repository consists of two distinct components: (1) a formal Python reference package (`src/secrethandle`) for offline state-machine verification, property testing, and math audits; and (2) the Android 14 on-device prototype (`device/b3-secrethandle/`) evaluated in Section **??**, comprising the privileged Binder service, notification listener broker, accessibility redactor, Agent SDK bridge, and microbenchmark suites.

Contents. The repository ships: (i) `src/secrethandle`: Python state-machine implementation, policy solvers, offline pytest suite, and deterministic synthetic-canary property checks; (ii) `device/b3-secrethandle`: Java and AIDL source code for `SecretHandleVaultService`, the `NotificationListener` broker, the `TYPE_ACCESSIBILITY_OVERLAY` compositor, and the in-framework `BenchmarkRunner.java` harness; (iii) `evaluation/`: Experiment drivers, MobileAgent v3.5 bridge configuration, attack harness, synthetic bank and attacker APKs, and log parsers for reproducing Table **??**.

One-command offline verification. From the repository root, with Python 3.10+ (3.11+ recommended):

```
python -m venv .venv && source .venv/bin/activate
python -m pip install -e '.[dev]'
make check
```

`make check` executes linting (`ruff`), type checks (`mypy`), unit test suites (`pytest`), and the synthetic-canary property checker. Running `make smoke` tests state-machine invariants over 6,400 synthetic attack runs (seed `20261017`) with zero leak anomalies.

On-device Android 14 experiment reproduction. To reproduce the empirical measurements in Section **??**:

```
# 1. Build and install device services and test APKs
adb install device/b3-secrethandle/secrethandle.apk
adb install evaluation/apks/com.legitimate.bank.apk
adb install evaluation/apks/com.attacker.rewards.apk
# 2. Run latency microbenchmarks via app_process
adb shell CLASSPATH=/data/local/tmp/bench.jar \
  app_process /data/local/tmp BenchmarkRunner
# 3. Execute n=30 agent attack evaluation trials
python evaluation/run_agent_matrix.py \
  --config evaluation/configs/android14_eval.yaml
```

Per-trial logs, latency dumps, and step trajectories are recorded in `results/trials.jsonl`. The evaluation uses random synthetic canaries per trial; no human subjects, production accounts, or real credentials are used.

Integrity and license. All configuration digests and random seeds are logged in `provenance.json` at initialization. The repository is released under the Apache License 2.0 with all source files, benchmark harnesses, and analysis scripts fully open-sourced.